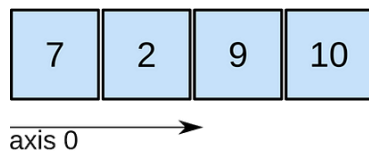


What is NumPy?

NumPy, is short for **Numerical Python**, it is an open-source Python library. **NumPy** is a library consisting of multi-dimensional arrays (matrices) and provides a wide range of mathematical functions for array operations. It is used in scientific computing, and in areas like data analysis, machine learning, etc.

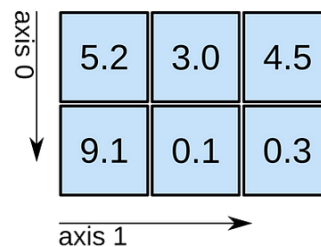
The **primary data structure in NumPy is the N-dimensional array** -- called an ndarray or simply an array. Every ndarray is a fixed-size array that is kept in memory and contains the same type of data such as integer or floating-point numbers

1D array



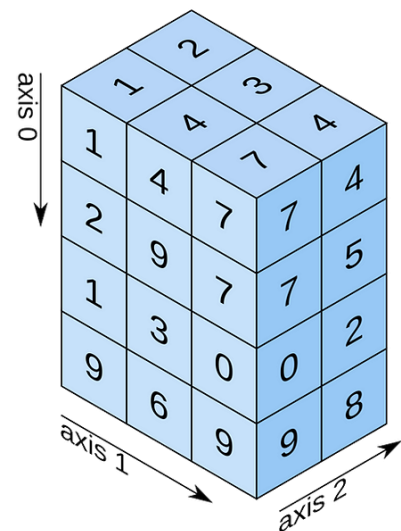
shape: (4,)

2D array



shape: (2, 3)

3D array



shape: (4, 3, 2)

NumPy array can contain strings. **NumPy is primarily known for numerical operations**, it can store various data types, including strings.

When creating a NumPy array with strings, the dtype will typically be set to a fixed-length string type (e.g., "<U10" for a Unicode string with a maximum length of 10 characters) or object for variable-length strings.

The elements of a NumPy array, or simply an array, are usually numbers, but can also be boolians, strings, or other objects.

Use of NumPy

- NumPy provides various math functions for calculations like addition, algebra, and data analysis.
- NumPy provides various objects representing arrays and multi-dimensional arrays which can be used to handle large data such as images, sounds, etc.
- NumPy also works with other libraries like SciPy (for scientific computing), Pandas (for data analysis), and scikit-learn (for machine learning).
- NumPy is fast and reliable, which makes it a great choice for numerical computing in Python.

NumPy Applications

- **Data Analysis:** For Data analysis, while handling data, we can create data (in the form of array objects), filter the data, and perform various operations such as mean, finding the standard deviations, etc.
- **Machine Learning & AI:** Popular machine learning tools like TensorFlow and PyTorch use NumPy to manage input data, handle model parameters, and process the output values.
- **Array Manipulation:** NumPy allows you to create, resize, slice, index, stack, split, and combine arrays.
- **Finance & Economics:** NumPy is used for financial analysis, including portfolio optimization, risk assessment, time series analysis, and statistical modelling.
- **Image & Signal Processing:** NumPy helps process and analyze images and signals for various applications.

- **Data Visualization:** NumPy independently does not create visualizations, but it works with libraries like Matplotlib and Seaborn to generate charts and graphs from numerical data.

What you need to Learn NumPy

You should have a basic understanding of computer programming terminologies. A basic understanding of Python and any of the programming languages is a plus.

Which Language is NumPy written in?

NumPy is primarily written in the following languages –

- C
- Python
- Fortran

Installing NumPy

The NumPy module is not included in the standard Python distribution earlier. However, it can be easily installed using the Python package installer pip –

➤ `pip install numpy`

Verifying Installation

To verify that NumPy is installed correctly, open a Python prompt and try importing it –

`import numpy`

If it is not installed, the following error message will be displayed –

Traceback (most recent call last):

File "<pyshell#0>", line 1, in <module>

`import numpy`

ImportError: No module named 'numpy'

Alternatively, you can import the NumPy package using the standard alias –

```
import numpy as np
```

If NumPy is properly installed, no error messages will be displayed. Otherwise, an ImportError will indicate that NumPy is not found.

Checking NumPy Version

```
import numpy as np
```

```
print(np.__version__)
```

Example: Create a One-dimensional Array

To create an array in NumPy use np.array function and just pass a list

```
import numpy as np
```

```
a = np.array([1, 2, 3])
```

```
print(a)
```

Example: Creating a Two-dimensional Array

```
import numpy as np
```

```
a = np.array([[1, 2], [3, 4]])
```

```
print(a)
```

NumPy Data Types

NumPy supports a much greater variety of numerical types than Python does. The following table shows different scalar data types defined in NumPy.

Basic Numerical Types:

Integers:

- int8, int16, int32, int64: Signed integers with varying bit lengths.
- uint8, uint16, uint32, uint64: Unsigned integers with varying bit lengths.

Floating-Point Numbers:

- float16, float32, float64: Floating-point numbers with varying precision.
- float128: Extended-precision floating-point numbers (precision may vary depending on the system).

Complex Numbers:

- complex64: Complex numbers represented by two 32-bit floats.
- complex128: Complex numbers represented by two 64-bit floats.

Other Data Types:

- bool: Boolean values (True or False).
- object: Python objects.

- string_, unicode_: Strings.
- datetime64: Date and time values.
- timedelta64: Differences between two datetime values.

refer to data types with one character, like i for integers, u for unsigned integers etc.

i - integer

b - boolean

u - unsigned integer

f - float

c - complex float

m - timedelta

M - datetime

O - object

S - string

U - unicode string

V - fixed chunk of memory for other type (void)

Checking the Data Type of an Array

The NumPy array object has a property called `dtype` that returns the data type of the array:

Example

Get the data type of an array object:

```
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr.dtype)
```

Example

Get the data type of an array containing strings:

```
import numpy as np

arr = np.array(['A', 'B', 'C'])

print(arr.dtype)
```

Creating Arrays With a Defined Data Type

We use the `array()` function to create arrays, this function can take an

optional argument: dtype that allows us to define the expected data

type of the array elements:

Example

Create an array with data type string:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4], dtype='S')
```

```
print(arr)
```

```
print(arr.dtype)
```

For i, u, f, S and U we can define size as well.

Example

Create an array with data type 4 bytes integer:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4], dtype='i4')
```

```
print(arr)
```

```
print(arr.dtype)
```

If a type is given in which elements can't be casted then NumPy will raise a ValueError.

Key Features of NumPy Data Types:

Fixed Size: NumPy data types have a fixed size in memory, which contributes to efficiency.

Memory Efficiency: NumPy's data types allow you to specify the exact memory usage needed for your data, unlike Python's dynamic types.

Platform Dependency: Some data types, like `intp` and `longdouble`, may have varying sizes depending on the system architecture.

Character Codes: NumPy data types can be referenced by character codes for backward compatibility with older packages.

Creating NumPy Array

We can create a NumPy array using various function provided by the Python NumPy library. Following are the functions using which we can create a NumPy array –

- `numpy.array()` Function
- `numpy.zeros()` Function
- `numpy.ones()` Function
- `numpy.arange()` Function

- `numpy.linspace()` Function
- `numpy.random.rand()` Function
- `numpy.empty()` Function
- `numpy.full()` Function

Unlike Python lists, NumPy arrays support element-wise operations and are more memory-efficient, making them useful for mathematical computations.

Creating NumPyArray Using numpy.array() Function

We can use the `numpy.array()` function to create an array by passing a Python list or tuple as an argument to the function.

`numpy.array()` function converts input data (like lists, tuples, etc.) into an ndarray (NumPy array).

Syntax

`numpy.array(object, dtype=None, copy=True, order='K', subok=False, ndmin=0, like=None)`

Example: Creating a 1D NumPy Array

In the following example, we are creating a 1-dimensional NumPy array from a list of integers using the `numpy.array()` function –

```
import numpy as np
```

```
my_list = [1, 2, 3, 4, 5]
```

```
my_array = np.array(my_list)
```

```
print("1D Array:", my_array)
```

output –

```
1D Array: [1 2 3 4 5]
```

Example: Creating a 2D NumPy Array

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print("2D Array:\n", arr)
```

Output –

2D Array:

[[1 2 3]

[4 5 6]]

Creating NumPyArray Using numpy.zeros() Function

We can also use the `numpy.zeros()` function for creating an array by specifying the desired shape of the array as a tuple or an integer.

`numpy.zeros()` function creates a NumPy array filled with zeros. It accepts the shape of the array as an argument and optionally the data type (`dtype`). By default, the data type is `float64`.

Syntax

`numpy.zeros(shape, dtype=float, order='C')`

Example

In this example, we are creating a NumPy array with 5 elements, all initialized to zero using the `numpy.zeros()` function –

```
import numpy as np

arr = np.zeros(5)

print(arr)
```

Output –

```
[0. 0. 0. 0. 0.]
```

Creating NumPyArray Using `numpy.ones()` Function

On the other hand, the `numpy.ones()` function creates an array where all elements are set to 1. It accepts three main parameters: shape, dtype, and order.

The shape parameter, which can be an integer or a tuple of integers, defines the dimensions of the array.

The dtype parameter specifies the desired data type of the array elements, defaulting to "float64" if not provided.

The order parameter determines the memory layout of the array, either row-major (C-style) or column-major (Fortran-style), with 'C' being the default.

Syntax

numpy.ones(shape, dtype=None, order='C')

Example: Creating 1D array of ones

In the example below, we are creating a 1 dimensional NumPy array with 3 elements, all initialized to one using the `numpy.ones()` function –

```
import numpy as np

arr = np.ones(3)

print(arr)
```

Output –

```
[1. 1. 1.]
```

Example: Creating 2D array of ones

In here, we create a 2 dimensional NumPy array with 4 rows and 3 columns, filled with ones, using the `np.ones()` function –

```
import numpy as np
```



```
array_2d = np.ones((4, 3))
```

```
print(array_2d)
```

Output –

```
[[1. 1. 1.]
```

```
 [1. 1. 1.]
```

```
 [1. 1. 1.]
```

```
 [1. 1. 1.]]
```

Example: Creating a Fortran-ordered array of ones

Now, we are creating a 2-dimensional NumPy array with 2 rows and 3 columns, filled with ones, using the `np.ones()` function with Fortran-style (column-major) order –

```
import numpy as np
```

```
array_F = np.ones((4, 3), order='F')
```

```
print(array_F)
```

Output–

```
[[1. 1. 1.][1. 1. 1.][1. 1. 1.][1. 1. 1.]]
```

Creating NumPyArray Using numpy.arange() Function

The `numpy.arange()` function creates an array by generating a sequence of numbers based on specified start, stop, and step values. It is similar to Python's built-in `range()` function.

Syntax

`numpy.arange([start, ]stop, [step, ]dtype=None, *, like=None)`

`start` (optional) – The starting value of the interval. Default is 0.

`stop` – The end value of the interval (not included).

`step` (optional) – The spacing between values. Default is 1.

`dtype` (optional) – The desired data type for the array. If not given,

NumPy interprets the data type from the input values.

Example

```
import numpy as np

# Create an array from 0 to 9

arr = np.arange(10)

print("Array using arange():", arr)
```

Example: Specifying Start, Stop, and Step

```
import numpy as np

# Create an array from 1 to 9 with a step of 2

arr = np.arange(1, 10, 2)

print("Array with start, stop, and step:", arr)
```

Creating NumPyArray Using numpy.linspace() Function

We can even use the `numpy.linspace()` function to create an array by specifying the start, stop, and the number of elements we want.

The array created by this function consists of evenly spaced values over a specified interval. The function takes parameters for start, stop, and the number of elements, and generates values that are evenly distributed between the start and stop values, inclusive. Following is the

Syntax

numpy.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0)

The `numpy.linspace()` function is particularly useful when you need a set number of points between two endpoints for plotting or numerical computations.

Example

In the example below, we are using `numpy.linspace()` function to create three arrays (`array1`, `array2`, and `array3`) with specified ranges and configurations.

The "`array1`" is created with 10 evenly spaced values from 0 to 5, inclusive. The "`array2`" consists of 5 values ranging from 1 to just under 2, excluding the endpoint. The "`array3`" is created with 5 values from 0 to 10, and returns both the array and the step size between consecutive values –

```
import numpy as np

array1 = np.linspace(0, 5, num=10)

print("array1:",array1)

array2 = np.linspace(1, 2, num=5, endpoint=False)

print("array2:",array2)

array3, step = np.linspace(0, 10, num=5, retstep=True)

print("array3:",array3)

print("Step size:", step)
```

Output–

```
array1: [0.      0.55555556 1.11111111 1.66666667 2.22222222 2.77777778
 3.33333333 3.88888889 4.44444444 5.      ]
```

array2: [1. 1.2 1.4 1.6 1.8]

array3: [0. 2.5 5. 7.5 10.]

Step size: 2.5

Creating NumPyArray Using random.rand() Function

Alternatively, we can use the `numpy.random.rand()` function for creating an array by specifying the dimensions of the array as parameters.

This function is used to create an array of specified shape filled with random values sampled from a uniform distribution over $[0, 1)$.

It accepts parameters for the dimensions of the array (like `numpy.random.rand(rows, columns)`), and generates an array of the specified shape with random values between 0 and 1. If no argument is provided, it returns a single random float value.

Syntax –

`numpy.random.rand(d0, d1, ..., dn)`

Example

In the following example, we are using `numpy.random.rand()` function to generate arrays of random floats with different dimensions –

```
import numpy as np
```

```
# Single random float
```

```
random_float = np.random.rand()
```

```
print("random_float:",random_float)
```

```
# 1D array of random floats
```

```
array_1d = np.random.rand(5)
```

```
print("array_1d:",array_1d)
```

```
# 2D array of random floats
```

```
array_2d = np.random.rand(2, 3)
```

```
print("array_2d:",array_2d)
```

```
# 3D array of random floats
```

```
array_3d = np.random.rand(2, 3, 4)
```

```
print("array_3d:",array_3d)
```

output

```
random_float: 0.5030496450079744
```

```
array_1d: [0.19476581 0.54430648 0.64571106 0.27443774 0.71874319]
```

```
array_2d: [[0.91141582 0.58847504 0.37284854]
```

```
[0.0715398 0.21305363 0.766954  ]]
```

```
array_3d: [[[0.7295106 0.1582053 0.91376381 0.14099229]
[0.6876814 0.19351871 0.18056163 0.61370308]
[0.42382443 0.6665121 0.42322218 0.11707395]]

[[[0.60883975 0.01724199 0.95753734 0.17805716]
[0.47770594 0.55840874 0.7375783 0.50512301]
[0.73730351 0.85900855 0.16472072 0.2338285 ]]]
```

Creating NumPyArray Using `numpy.empty()` Function

We can create a NumPy array using the `numpy.empty()` function by specifying the shape of the array as parameters.

This function initializes an array without initializing its elements; the content of the array is arbitrary and may vary. It is useful when you need an array of a specific size and data type, but you intend to fill it later with data.

Syntax –

`numpy.empty(shape, dtype=float, order='C')`

Unlike `numpy.zeros()` function and `numpy.ones()` function, which initialize array elements to zero and one respectively, the `numpy.empty()` function does not initialize the elements. Instead, it allocates the memory required for the array without setting any values.

Example

In this example, we are using `numpy.empty()` function to create a 2-dimensional array (`empty_array`) with 2 rows and 3 columns –

```
import numpy as np

empty_array = np.empty((2, 3))

print(empty_array)
```

output

```
[[1.13750619e-313 0.00000000e+000 0.00000000e+000]
 [0.00000000e+000 0.00000000e+000 0.00000000e+000]]
```

Unlike `numpy.zeros()`, this function initializes the array with uninitialized values, which could be any random data left in memory, making it suitable for cases where immediate initialization is not required.

Creating NumPyArray Using numpy.full() Function

Using the numpy.full() function, we can create an array with a desired shape and set all the elements in it to a specific value.

Syntax –

numpy.full(shape, fill_value, dtype=None, order='C')

Example

In the following example, we are using the numpy.full() function to create a 2-dimensional array with dimensions 2x3, filled entirely with the value 5 –

```
import numpy as np

array1 = np.full((2, 3), 5)

print(array1)
```

Output –

```
[[5 5 5]
```

```
[5 5 5]]
```

Functions Used for Creation of NumPy Arrays

In the NumPy module, there are various ways to create NumPy arrays that includes, basic creation methods, creation by reshaping and modifying data, creation using sequences, and creation using random functions. Following are the different functions used to create NumPy arrays –

Basic Array Creation

Following are the basic functions for creation of array –

Functions	Description
array()	used to create a numpy array
asarray()	Convert the input to an array
asanyarray()	Convert the input to an ndarray, but pass ndarray subclasses through
copy()	Return an array copy of the given object

Array Creation with Specific Shapes and Data

Following are the functions used to create an array with specified shapes and data –

Functions	Description
-----------	-------------

<code>zeros()</code>	Return a new array of given shape and type, filled with zeros
<code>ones()</code>	Return a new array of given shape and type, filled with ones
<code>empty()</code>	Return a new array of given shape and type, without initializing entries
<code>full()</code>	Return a new array of given shape and type, filled with <code>fill_value</code>

Array Creation with Sequences

Following are the functions used to create an array with sequences –

Functions	Description
<code>arange()</code>	Return evenly spaced values within a given interval
<code>linspace()</code>	Return evenly spaced numbers over a specified interval.
<code>logspace()</code>	Return numbers spaced evenly on a log scale.

Random Arrays

Following are the random functions to create an array –

Functions	Description
<code>random.rand()</code>	Random values in a given shape
<code>random.randn()</code>	Return a sample from the standard normal distribution
<code>random.randint()</code>	Return random integers from low (inclusive) to high (exclusive)
<code>random.random()</code>	Return random floats in the half-open interval [0.0, 1.0)
<code>random.choice()</code>	Generates a random sample from a given 1-D array

NumPy Array Indexing - Access Array Elements

You can access an array element by referring to its index number.

The indexes in NumPy arrays start with 0, meaning that the first element has index 0, and the second has index 1 and so on

Example

```
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr[0])
```

Example

Get third and fourth elements from the following array and add them.

```
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr[2] + arr[3])
```

Access 2-D Arrays

To access elements from 2-D arrays we can use comma separated integers representing the dimension and the index of the element.

Think of 2-D arrays like a table with rows and columns, where the dimension represents the row and the index represents the column.

Example

Access the element on the first row, second column:

```
import numpy as np

arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('2nd element on 1st row: ', arr[0, 1])
```

Access 3-D Arrays

To access elements from 3-D arrays we can use comma separated integers representing the dimensions and the index of the element.

Example

Access the third element of the second array of the first array:

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])

print(arr[0, 1, 2])
```

arr[0, 1, 2] prints the value 6.

The first number represents the first dimension, which contains 2 arrays:

[[1, 2, 3], [4, 5, 6]]

and:

[[7, 8, 9], [10, 11, 12]]

Since we selected 0, we are left with the first array:

[[1, 2, 3], [4, 5, 6]]

The second number represents the second dimension, which also

contains two arrays:

[1, 2, 3]

and:

[4, 5, 6]

Since we selected 1, we are left with the second array:

[4, 5, 6]

The third number represents the third dimension, which contains three

values:

4

5

6

Since we selected 2, we end up with the third value: 6

Negative Indexing

Use negative indexing to access an array from the end.

Example

Print the last element from the 2nd dim:

```
import numpy as np

arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('Last element from 2nd dim: ', arr[1, -1])
```

NumPy Array Slicing

Slicing in python means taking elements from one given index to another given index.

We pass slice instead of index like this: **[start:end]**.

We can also define the step, like this: **[start:end:step]**.

- If we don't pass start its considered 0
- If we don't pass end its considered length of array in that dimension
- If we don't pass step its considered 1

Example

Slice elements from index 1 to index 5 from the following array:

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5, 6, 7])  
  
print(arr[1:5])
```

Note: The result includes the start index, but excludes the end index.

Negative Slicing

Use the minus operator to refer to an index from the end:

Example

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4, 5, 6, 7])  
  
print(arr[-3:-1])
```

STEP

Example

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[1:5:2])
```

Example

Return every other element from the entire array:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[:2])
```

Slicing 2-D Arrays

Example

From the second element, slice elements from index 1 to index 3

```
import numpy as np

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

print(arr[1, 1:4])          # Remember that second element has index 1.
```

Example

From both elements, return index 2:

```
import numpy as np

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

print(arr[0:2, 2])
```

Example

From both elements, slice index 1 to index 4 (not included), this will

return a 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])

print(arr[0:2, 1:4])
```